

Deep Learning 1 - CNNs, Transformers and GNNs

Matúš Halák - 13626574 (UvA Student ID)

November 2025

1 Properties of Convolutional Neural Networks

Question 1.1a, c(i) Here, we study convolution in the context of simple synthetic image problem with two classes. Each image contains a red and green square on a black background. Label 0 corresponds to the case when the red square appears left of the green square, and label 1 corresponds to the case when the red square appears right of the green square. Crucially, we have an interesting train, validation and test split. All the examples in the training and validation set for a given class occur on one half of the screen (label 0 - top, label 1 - bottom), and all the examples in the test set for the same class occur on the opposite vertical half of the screen (label 0 - bottom, label 1 - top). This setup allows the model to achieve 100% accuracy on the training and validation set by classifying the images using the incorrect top-bottom half rule (perfectly separating classes in the training set) instead of the true rule about the relative position of the red and green squares. In this case, the test accuracy would automatically be 0%, because the model would always predict the label based on the wrong half of the screen. Results of applying the various convolution types in both Net1 and Net2 can be seen in Table 1 and Figure 1. The only difference between Net1 and Net2 is that Net1 uses Max Pooling while Net2 uses a Linear layer to downsample before the final linear layer.

Conv	Dataset	Net1	Net2
valid	validation	100.00 (0.00)	100.00 (0.00)
valid	test	0.09 (0.11)	0.00 (0.00)
circular	validation	99.59 (0.33)	100.00 (0.00)
circular	test	83.07 (3.21)	0.00 (0.00)
replicate	validation	98.01 (0.72)	100.00 (0.00)
replicate	test	94.10 (2.11)	0.00 (0.00)
sconv	validation	98.96 (0.18)	100.00 (0.00)
sconv	test	6.51 (0.31)	0.00 (0.00)
fconv	validation	88.09 (0.95)	100.00 (0.00)
fconv	test	89.06 (0.39)	0.00 (0.00)
reflect	validation	100.00 (0.00)	100.00 (0.00)
reflect	test	0.00 (0.00)	0.00 (0.00)

Table 1: **Classification accuracy per Convolution Type and Network.** Each cell shows mean accuracy (standard deviation) across 10 random initializations for the corresponding network and dataset.

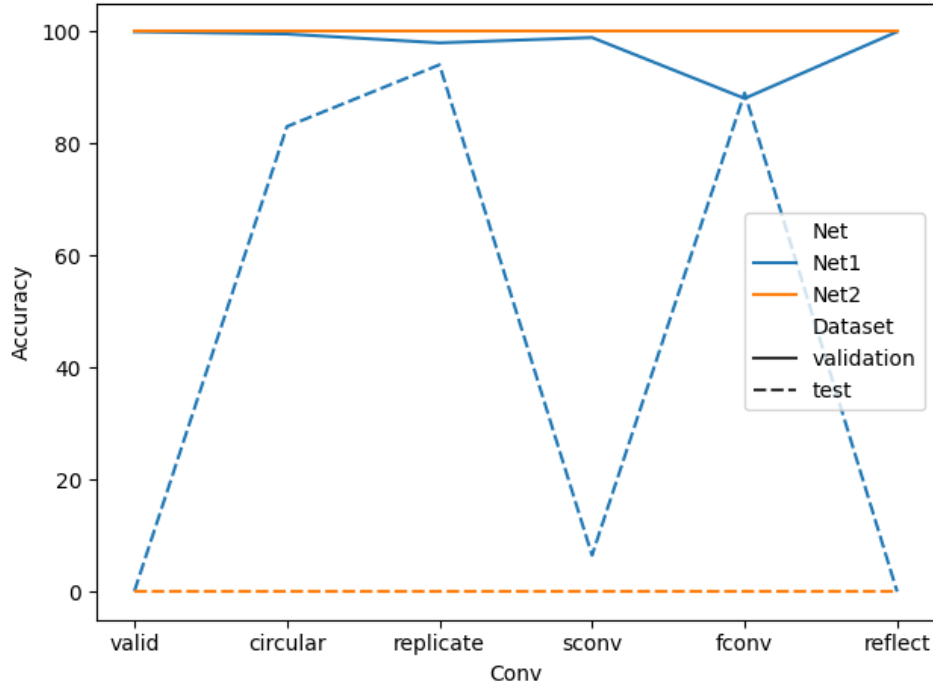


Figure 1: **Classification accuracy per Convolution Type and Network.**

Question 1.1b(i) Above we used a number of different convolution types and saw quite dramatic effects on the model performance. Here we explain what the various convolution types correspond to. We note that our network has four convolutional layers, followed by two linear layers, where the final linear layer returns two logits for the two possible classes. Each conv layer uses 3x3 kernels, and while the conv layer 1 uses a stride 1, conv layers 2-4 use stride 2.

- *valid* convolution does not perform padding. Therefore, with each convolutional layer using a 3x3 kernel, the image shrinks by 2 pixels in both height and width.
- *sconv* performs 'same' zero padding, by adding 1 pixel on each side. This maintains the "same" dimensions of the image as it goes through convolutional layers with 3x3 kernels.
- *fconv* performs 'full' zero padding by adding 2 pixels on each side. This actually grows the image by 1 pixel on each side with each convolution (since one of the added pixels on each side disappears through the convolution with the 3x3 kernel). The idea behind this approach is that each pixel is fully considered, not just when the kernel is centered on it, but also as an outer edge in a convolution of all its neighbors.
- *circular*, *replicate*, *reflect* all pad 2 pixels on each side similar to *fconv*. Unlike the previous convolution types, they do not use zero padding but instead re-use pixel values from the image. *circular* pads using pixels from the opposite side of the image (wraps 2 border pixels around to other side). *replicate* pads by replicating the final border pixel over the 2 padded pixels. *reflect* pads by reflecting or mirroring the 2 pixels nearest the image border across the border to the 2 padded pixels.

The reason for the $valid < sconv < fconv$ test set accuracy order, is that the spatial dimensions of conv layer output activation maps produced with *valid*, *sconv* and *fconv* also follow this order, with the final conv layer output for the three conv types being (2x2, 4x4, 6x6). Because the classification task depends on the position of the small red and green squares relative to each other, and we want to make sure there is enough resolution in the convolution output that the two relative position cases can be distinguished. Furthermore, after the first conv layer, stride 2 sampling is used, which halves the spatial dimension with each conv layer. Since *valid* shrinks the image in each iteration, combined with stride 2, the individual boxes may vanish or overlap before the max-pooling and final classification layers, leading to poor accuracy when relative horizontal information is needed in the test set. *sconv* is better because information about the individual squares survive more stride 2 conv layers, and *fconv* is the best because the full padding provides most protection against the stride 2 downsampling with each conv layer.

Question 1.1b(ii) Test accuracy of *reflect* is worse than *fconv*, because *reflect* introduces artifacts that only happen at when the boxes are near edges and never in the center of the image. Because all training instances of class 0 and class 1 only ever happen on top or bottom halves, the network is also trained to associate the specific edge-artifacts with the class. At test time, opposite or new edge-artifacts occur that were not seen in the training set, and this confuses the network. On the other hand *fconv* zero padding behaves consistently across training and test set, and therefore leads to better performance.

Question 1.1b(iii) Test accuracy of *replicate* is better than *fconv*, because for cases where a colored box is at the edge, *replicate* extends the edge with the colored pixels for that box, making it easier to maintain information about the border-touching box throughout convolutional layers with stride 2 subsampling. *fconv* arbitrarily zeroes anything beyond the original image and does not continue the image intensity at the border, which may lead to strange discontinuities or shrink the boxes more than necessary after successive stride 2 convolutions.

Question 1.1c(ii) The test accuracies for all conv types in 'Net 2' go to 0% (Table 1), which means test accuracy for 'Net 2' is $\leq$ test accuracy for 'Net 1' across conv types.

Question 1.1c(iii) As mentioned before, the only difference between Net1 and Net2 is that Net1 uses Max Pooling while Net2 uses a Linear layer to downsample the spatial resolution to 1x1 before the final linear layer, which takes 64 down-sampled 1x1 activation maps, and produces logits for class 0 and class 1.

The reason why it is better to do max pooling (Net 1) is that this still considers all information from all spatial regions of the convolutional activation maps, and feeds in the max for each activation map across all spatial locations to the final linear layer. This means, that even if in the training set examples only ever occur on the top and produce high activation only on the top pixels, if the same situation and high activation occurs at test time in the bottom part of the image, the max pooling will choose the same highest activation, regardless of the fact that in the training set it never occurred in the bottom half of the screen. In other words, downsampling convolution output with max pooling preserves translation-invariance of convolutions, which is exactly what we need for the network to perform on the test set for this specific task.

On the other hand, learning a linear layer to downsample the spatial dimensions to 1x1 means that only weights for connections seen and used in training will be learned. This will lead exactly to the scenario which we observe, where accuracy on the validation part of the training set is perfect but accuracy on the test set is 0%. This is because the weights for the downsampling linear layer likely converged to producing 1x1 activation maps based on whether the squares were in the top or bottom half of the screen during training. This leads to exactly the opposite and wrong predictions at test time, even though this was the optimal linear layer to learn for the training and validation set. From this we can conclude that downsampling convolution output with a linear layer destroys translation invariance of the convolutions. Since the test performance relies fully on translation invariance of our network, this approach kills the test performance.

Question 1.2: CNN Rotation invariance on CIFAR10 Here we trained another CNN (3 convolutional layers followed by max-pooling and 2 fully-connected layers) on the CIFAR10 dataset (Figure 2) and investigated rotation invariance in this network.

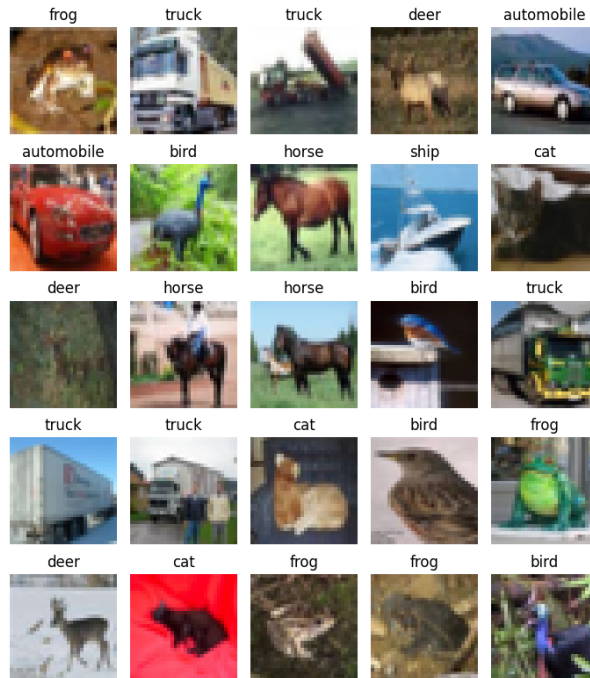


Figure 2: Example CIFAR10 images and labels.

We observe that a network trained on the default "straight" CIFAR10 dataset (Figure 2) performs poorly when tested on a rotated test set (Figure 3), with best accuracy $\approx 72\%$ at the unrotated angles (0 and 360). We note accuracy peaks at multiples of 90-degree rotations, as in those cases, all pixels in the image are available to be used for classification. At other rotation angles, some pixels may fall out of the image frame while other pixels in the image frame may simply be blank, which reduces the amount of information the network has to perform the classification. Furthermore, the accuracy peak at 180 degree rotation may stem from the fact that for images that are roughly symmetric horizontally and vertically, more relevant information is preserved with a 180 degree rotation than other rotations. Nevertheless, across rotation angles, the accuracy on rotated test images is $\leq$ half the accuracy on unrotated test images, indicating that CNN's are not intrinsically rotation invariant.

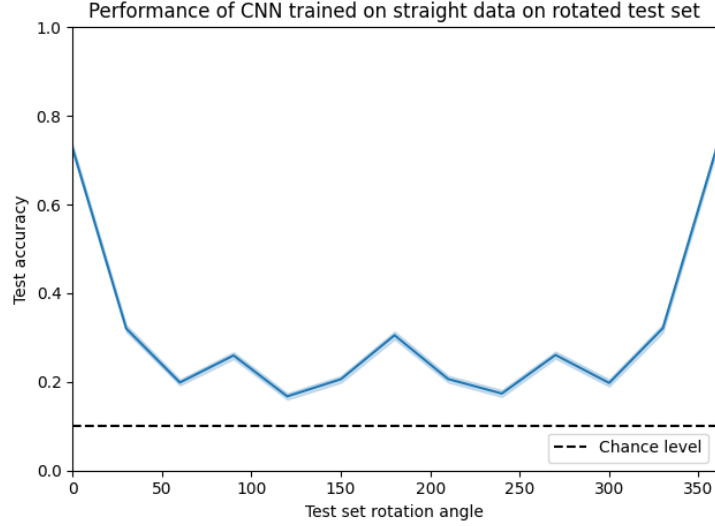


Figure 3: Classification of rotated test images by CNN trained on straight images.

Next, we trained the same CNN, on an augmented "rotated" CIFAR10 dataset, where each image was randomly rotated by anywhere between 0 and 360 degrees (Figure 4).

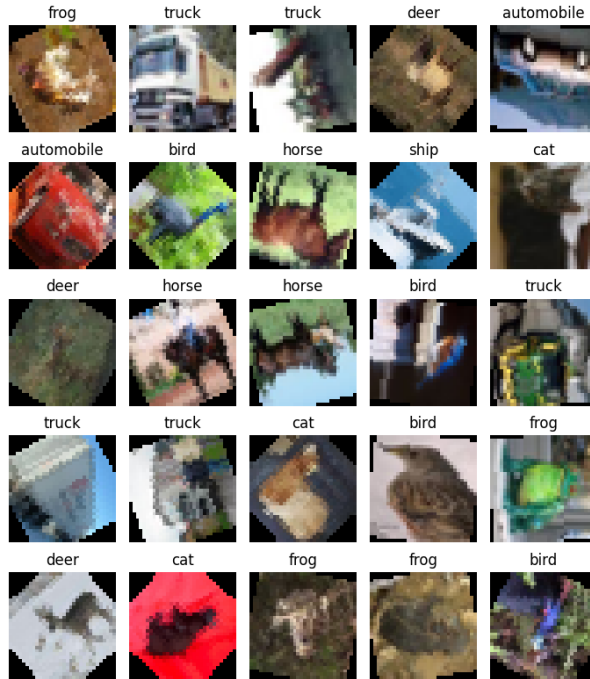


Figure 4: Example rotation-augmented CIFAR10 images and labels.

Crucially, the CNN trained on the augmented CIFAR10 dataset showed equal test accuracy across all possible rotations of test data (Figure 5) - in other words, with the simple augmentation of the training set, the CNN showed rotation invariance on the test set. The overall test accuracy achieved ($\approx 55\%$) was noticeably lower than the best accuracy achieved for the unrotated images by the original network ($\approx 72\%$). This was expected since we trained both networks for the same number of epochs with the same hyperparameters, but the rotation-augmented CNN had to learn a much harder task - namely classifying training images regardless of their random rotation in each epoch. The

fact that the same CNN architecture trained on the same dataset simply augmented by random rotations generalizes so well across all possible rotations of never seen test images indicates that rotation invariance can be successfully integrated into CNNs by randomly rotating the training images. This is in contrast with translation invariance which is an intrinsic property of convolutional layers. Combined together, the translation and rotation invariance properties make CNNs very powerful tools for object detection and classification in images.

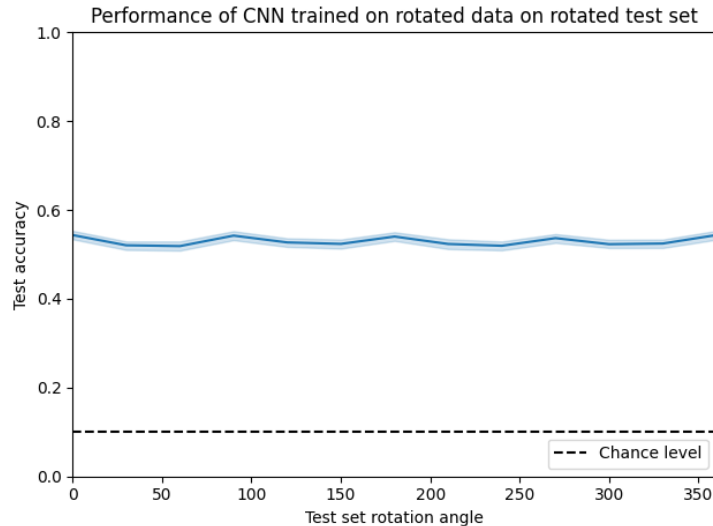


Figure 5: Classification of rotated test images by CNN trained on rotated images.

2 Attention, Transformers, and LLMs

2.1 Attention and Transformer theory

Question 2.1(a): Challenges with long input sequences in Transformers For an input sequence length T indicating the number of tokens in the input sequence, each attention head inside a transformer produces a $T \times T$ matrix of attention activations. This attention matrix is obtained via a (scaled) matrix multiplication between $Q \in \mathbb{R}^{T \times d}$ and $K^T \in \mathbb{R}^{d \times T}$, which is a $\mathcal{O}(T^2)$ operation. In other words, the computational complexity of each attention operation in a Transformer scales quadratically with input sequence length, because every token interacts with every other token (even though decoder blocks only consider previous tokens, the complexity of attention remains quadratic). This quadratic scaling imposes a limit on sequence length, since the computational complexity quickly gets out of hand with longer input sequences.

A possible way to address this problem is to relax the requirement that every token interacts with every other token. Namely, we can impose some structure on the attention operation, so that only nearby tokens interact (similar to convolution). To not fully discard interactions between all tokens, we could introduce a few placeholder tokens to each sequence which absorb global interactions, while forcing all other tokens in the sequence to interact more locally.

Question 2.1(b): Receptive fields in Transformers vs CNNs In CNNs, receptive fields are local and they grow in size at deeper levels of the network. Thus, only neurons in deep layer of CNNs are able to use information from the entire input, while all neurons in the shallower layers can only process local neighborhoods in the input. With growing input size the depth of the CNN necessary to have neurons able to use information from the entire image increases. In contrast, neurons at the end of each attention layer in a Transformer have receptive fields covering the entire input. This makes Transformers much better equipped to capture long-range dependencies, since neurons across all layers of the transformer are always working with information about the entire input.

Question 2.1(c): Scaled dot product attention Scaled dot product attention is defined by the following equation:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where the output of $QK^T \in \mathbb{R}^{T \times T}$. Because this matrix multiplication sums over the d_k dimension of both Q and K , with increasing embedding dimension d_k , this increasingly large sum would enter the softmax. This is problematic because in a softmax over very large values the largest value completely dominates the distribution (because of the exponentials in the softmax). Consequently, softmax over very large values has vanishing gradients, since small changes to the vector of large values barely change the outcome where the largest values still dominates. Dividing by $\sqrt{d_k}$ effectively normalizes the QK^T similarity scores so that the softmax operates in a good range.

Question 2.1(d): Benefit of Multi-head attention When using multiple smaller attention heads (Multi-head attention, MHA), the Transformer considers the input sequence through linear mappings for queries, keys and values learned separately for each head. In other words, the Transformer considers several projections of the entire input in parallel. This means that the different scaled dot product attention outputs of the different heads are analogous to different activation maps produced by different learned kernels in a CNN. Thus, in MHA each head can learn to detect different relevant and complementary "features" of the sequence in parallel, which would not be the case with Single-head attention (SHA), which would only find one "activation map" over the input sequence. Therefore, the main advantage of using MHA in comparison to SHA with the same computational cost is the greater representational power of MHA.

2.2 CausalSelfAttention implementation

2.3 The Transformer block

Question 2.3(c): Transformer MLP width vs depth There are several reasons why transformers use a wide two-layer MLP after MHA instead of a deep MLP with less neurons per layer. Firstly, a deep MLP need to be processed sequentially - the deeper the MLP, the more sequential non-parallelizable steps are necessary. On the other hand, a wide linear layer can be processed just as fast as a narrow one on modern GPUs. Therefore, a very wide 2-layer MLP is a going to be faster than a deep MLP with the same number of parameters, which is important, because the MLP needs to be applied to each token separately. Secondly, the MLP in a Transformer block serves to enrich the embedding representation of individual tokens, which is best achieved by, expanding the embedding dimension, nonlinearly transforming the expanded embedding, and projecting the nonlinearly transformed token embedding back down to the embedding dimension. This is precisely what a very wide two-layer MLP does. Furthermore, Transformer blocks are usually stacked and so it is better to utilize depth across Transformer blocks than within a the MLP of a single Transformer block.

2.4 Positional embeddings

Question 2.4(a): What happens without positional embeddings? Because the attention in the transformer is inherently permutation equivariant (due to the fact that each token attends to every other token, regardless of their order), without positional embeddings each token has no information about its position in the sequence. In other words, the token embeddings of the same token appearing at the beginning, middle and end of the sequence are the same. However, we might want to treat each token differently based on where in the sequence it occurs, and for this reason we add positional embeddings to the token embeddings, by learning an embedding lookup table that for each possible position in the sequence returns an embedding vector with the same dimension as the token embedding vectors. This is especially important in language where a small change in the order of words / tokens can drastically change the meaning of the sentence.

Question 2.4(b): Absolute versus relative position embeddings in LLMs and Transformers A disadvantage of absolute position embeddings is that they assume that each input to the model has a well-defined beginning and ending, between which the absolute word/token information provides useful information. However, with LLMs the training (and testing) input can be any sequence scraped from the internet, which might start in the middle of the sentence and continue across multiple unrelated sentences. In such cases, absolute position information is less informative (since the beginning of the sequence may actually be the end of a sentence etc.) than information about the relative position of individual pairs of words/tokens in the sentence. For instance tokens within a certain relative distance to each other may be more relevant for each other. This means that relative position embeddings can be most effectively utilized by modifying the key and query vectors before computing the attention scores, so that query-key attention scores depend not only on the token embedding similarities between query and key tokens but also on the relative distance between the query and key tokens.

2.5 GPT forward implementation

2.6 RoPE implementation

2.7 Pre-training the LLM on Fairy Tales by Brothers Grimm

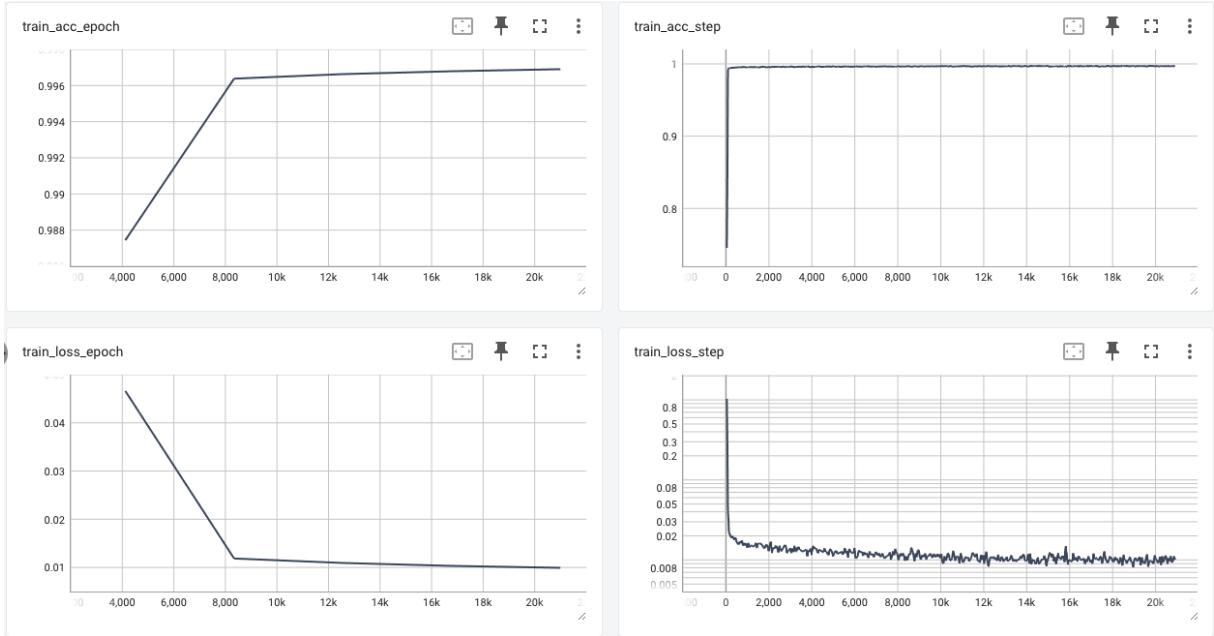


Figure 6: GPT2-mini training accuracy and loss curves per epoch and update step.

Figure 6 shows the results of pre-training a GPT2-mini model (default parameters) on next-token prediction of the Fairy Tales by Brothers Grimm for 5 epochs. We notice that the loss drops dramatically within the first two epochs, after which the model improves only incrementally. Already after the first epoch of training, the model achieves 98.8% training accuracy on average.

2.8 Testing the LLM-generated output

Question 2.8(b) Testing accumulated knowledge of model Here we perform a few tests on the pre-trained model. First, since the model was pre-trained on Fairy tales, we test what responses it outputs for fairy-tale like prompts versus unrelated prompts. For a fairly typical prompt that could easily occur in the fairy tales *"The princess looked at the knight and "*, the model (with top-k sampling $k = 5$) produced locally consistent, but globally nonsensical sentences made up of real words, including: *"The princess looked at the knight and said: 'Be whole tailor and sell this bake you, till you are takes, and you"*, where it correctly placed the *"and said"* phrase and followed it up with a reasonably-sounding statement. This is not surprising, since the model probably encountered this sentence structure many times during pre-training. Testing with the same prompt, but this time with top-p sampling and $p = 0.6$, led to generally better results, with some more *"and said"* phrases, but also other more logical continuations of the sentence. Keeping the same prompt, and same top-p sampling set-up, raising the temperature from 1 to 5 led to completely nonsensical results which did not consist of words such as: *"The princess looked at the knight and*

o‘CendbyCy:Y?Sm,rWY,KMFO*DDexM!TEYUNTLLeWiG.Y; yGY-s.1,PLuey’y. ‘IRs LyH’in r”. On the other hand, decreasing the temperature from 1 to 0.5 led to by far the best (most locally and globally consistent) results including *”The princess looked at the knight and she courted him and said: ‘I will be the mouse of the beautiful wife in the p”* and *”The princess looked at the knight and she said: ‘I will give you and the princess will be sea!’ ‘Ah!’ said the mous”*. We notice that decreasing the temperature indeed led to even more frequent occurrence of *”and said”* phrases, indicating that this was probably the most likely follow-up phrase based on the fairytale corpus. However, within the *”and said”* phrases and also other continuations of the prompt there remained considerable variety. Keeping the ‘best’ setting with top-p sampling and temperature 0.5, we finally tested a prompt that certainly did not occur in the Fairytale pre-training *”The corpus callosum is”*, which more or less broke the model (although still not as much as setting a high temperature) and led to outputs like *’The corpus callosum issippiiiiiiiiiiiiiiii THE SNOW-WART WELE See iin’*. These results show that the nature and variety of the pre-training data heavily dictates the zero-shot abilities of the model. Even if the zero-shot performance on prompts from similar distribution to pre-training is impressive, novel out-of-distribution prompts may confuse the model to the extent that it starts producing both locally and globally nonsensical predictions.

2.9 Transformer optimization

Question 2.9(b): Flash Attention A major strength of Flash Attention is that its output is equivalent to the output of the original Scaled dot-product attention how we implemented, but Flash attention computes this result much faster and more memory-efficiently. This means that for anyone simply wanting to use the (multi-head) attention layer without tinkering inside it, there is no reason not to use Flash attention.

Flash attention achieves the significant speed-ups through viewing the entire attention block (QK^T dot product similarity, softmax, attention score dot product with values) as a single operation to be optimized. Given the Q, K, V matrices, Flash-attention pre-computes the output values in a parallelized, fused operation (kernel) on the GPU SRAM that does not require intermediate results to be written and read from the slower GPU HBM (which turns out to be a significant performance bottleneck). To do this, it splits the output of the attention layer into blocks and in parallel computes the attention scores of the individual blocks. While softmax makes a naive implementation of this impossible (each value in each row depends on all other values in the row), the authors manage to decompose the softmax computation into blocks and are able to join the individual blocks in the end to produce results identical to normal attention layer.

Question 2.9(c): Downsides of large batch size Large batch size can lead to getting stuck in bad local minima and worse generalization. With very large batch size, the optimization approaches more deterministic full gradient descent and starts losing stochastic nature that helps (minibatch) SGD escape local minima. This may encourage the model to converge quickly to sharp local minima that generalize poorly. Furthermore, a large batch size places a demand on memory usage, effectively limiting model size, context length and other memory-intense parts of training and inference which might be more important.

3 Graph Neural Networks

3.1 Adjacency matrices

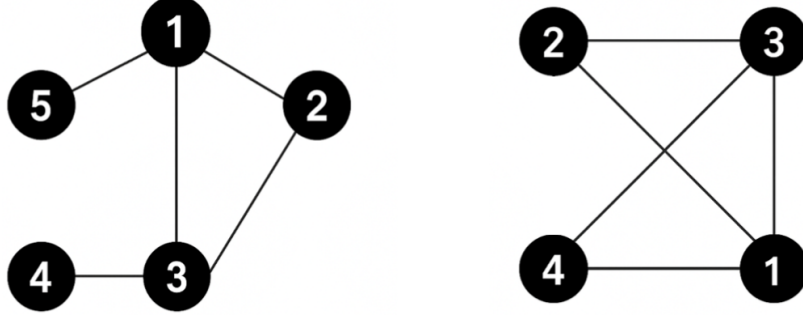


Figure 7: Two example graphs.

Question 3.1(a): Adjacency matrices The adjacency matrix of the left graph A_L from Figure 7 (represented as a nested list) is:

$$A_L = [[0, 1, 1, 0, 1], [1, 0, 1, 0, 0], [1, 1, 0, 1, 0], [0, 0, 1, 0, 0], [1, 0, 0, 0, 0]]$$

The adjacency matrix of the right graph A_R from Figure 7 is:

$$A_R = [[0, 1, 1, 1], [1, 0, 1, 0], [1, 1, 0, 1], [1, 0, 1, 0]]$$

Question 3.1(b): Squared adjacency matrix We write down the squared adjacency matrix of the left graph from Figure 7.

$$A_L^2 = A_L A_L = [[3, 1, 1, 1, 0], [1, 2, 1, 1, 1], [1, 1, 3, 0, 1], [1, 1, 0, 1, 0], [0, 1, 1, 0, 1]]$$

The diagonal elements of A_L^2 represent the neighborhood size or degree of the individual nodes. The off-diagonal elements of A_L^2 represent the number of common neighbors between any two nodes. This is equivalent to another perspective which generalizes better to arbitrary n , which is that $\{A^n\}_{uv}$ gives the number of possible walks from node u to node v in n steps. In that light, the reason why the diagonal of A_L^2 represents the neighborhood size of each node is because for each immediate neighbor, it is possible to go from the node to the neighbor and back to the node itself in two steps, and thus the number of possible two-step walks from the node to itself is exactly the number of immediate neighbors. Similarly, a common neighbor is one that two nodes are both immediately connected to. And therefore the number of two-step walks from one node to another is simply the number of common neighbors between those two nodes. With higher n , this concept extends to the number of possible n -step walks between any two nodes.

Question 3.1(c): Graph Laplacian The Laplacian matrix (Graph Laplacian) L is defined as:

$$L = D - A$$

where D is the degree matrix of the graph and A is the adjacency matrix of the graph.

For the right graph from Figure 7, we have

$$D_R = \text{diag}(3, 2, 3, 2) \quad \text{and} \quad A_R = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 \end{bmatrix}$$

Thus,

$$L_R = D_R - A_R = \begin{bmatrix} 3 & -1 & -1 & -1 \\ -1 & 2 & -1 & 0 \\ -1 & -1 & 3 & -1 \\ -1 & 0 & -1 & 2 \end{bmatrix}$$

We notice that the diagonal entries contain the degree (number of neighbors) of each node, while the off-diagonals include -1 for each neighboring node and 0 for unconnected nodes. This means that the effect of matrix multiplying a node feature matrix X by the Laplacian matrix L is to for each node, compute the sum of the differences between the node's features and the features of the node's neighbors. The nodes that change the most with this operation ($F = LX$) are nodes with the most neighbors and the largest feature differences relative to their neighbors. For those nodes, the most differences between their own and their neighbors' features will be computed and summed. In this case, in absence of information about feature similarities between nodes, we can only reason based on the neighborhood sizes. Both nodes 1 and 3 have 3 neighbors, so the features of nodes 1 and 3 would be most affected.

3.2 Graph convolution

For an undirected graph with N nodes $\{v\}_1^N$, each associated with a d -dimensional embedding h_v , we have the following graph convolution layer:

$$h_v^{(l+1)} = \sigma \left(W^{(l)} \sum_{u \in \mathcal{N}(v)} \frac{h_u^{(l)}}{|\mathcal{N}(v)|} + B^{(l)} h_v^{(l)} \right) \in \mathbb{R}^{d^{(l+1)}}$$

Question 3.2(a): Effect of separate self-connection term The self-loop term $B^{(l)} h_v^{(l)}$ corresponds to a learned projection $B^{(l)} \in \mathbb{R}^{d^{(l+1)} \times d^{(l)}}$ of each node's own features from the previous layer dimension to the next layer dimension. The importance of this term is that it gives each node in each layer some neighborhood-agnostic embedding signal which can then be enhanced or dampened by the neighborhood aggregation. Without this term, each node's embeddings would depend solely on the learned projection $W^{(l)}$ of the aggregated features across the node's neighbors, and the individual nodes would convey less unique information.

Question 3.3(a): Matrix version of graph convolution Here we derive the matrix formula for graph convolution. Our goal is to obtain the matrix $H^{(l+1)} \in \mathbb{R}^{N \times d^{(l+1)}}$, containing the nonlinear transformations of the node embeddings $h_v^{(l+1)}$ as its N rows.

First, we denote the neighborhood of node v with $\mathcal{N}(v)$, and denote the size of the node's neighborhood (the node's degree) with $|\mathcal{N}(v)|$. The degree matrix D of a graph with N nodes is a diagonal matrix defined as:

$$D = \text{diag}(|\mathcal{N}(v_1)|, \dots, |\mathcal{N}(v_N)|) \in \mathbb{R}^{N \times N}$$

Second, we identify parts of the graph convolution equation that require re-writing. The nonlinearity σ is applied element-wise, and therefore poses no issue for a matrix

formulation. Similarly, the learned linear projections $B^{(l)}, W^{(l)} \in \mathbb{R}^{d^{(l+1)} \times d^{(l)}}$ can be remain and be shared between nodes and be applied repeatedly to each node's embedding dimension. In fact the self-connection learned projection can immediately be expressed for all nodes' features $H^{(l)} \in \mathbb{R}^{N \times d^{(l)}}$ in one matrix multiplication $H^{(l)} B^{(l)T} \in \mathbb{R}^{N \times d^{(l+1)}}$. Therefore the only term that requires re-writing is the sum over features of neighboring nodes, normalized by the neighborhood size.

At this point we recognize that the adjacency matrix A picks out for each node only the nodes in its neighborhood. Therefore, matrix multiplying the adjacency matrix with the node feature matrix $H^{(l)}$ returns a $AH^{(l)} \in \mathbb{R}^{N \times d^{(l)}}$ matrix where each row corresponds to the sum over that node's neighboring nodes' features.

$$[AH^{(l)}]_{v,:} = \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$$

The only remaining step is to normalize this sum over neighboring features by the neighborhood size $1/|\mathcal{N}(v)|$. This can be expressed easily by matrix multiplying with the inverse of the diagonal degree matrix, which is just

$$D^{-1} = \text{diag}\left(\frac{1}{|\mathcal{N}(v_1)|}, \dots, \frac{1}{|\mathcal{N}(v_N)|}\right) \in \mathbb{R}^{N \times N}$$

which for each node gives a an average over the neighborhood features

$$[D^{-1}AH^{(l)}]_{v,:} = \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} h_u^{(l)}$$

Finally, as discussed above, the linear projection of aggregated neighborhood features $W^{(l)}$ is shared between nodes and can be repeatedly applied over the embedding dimension for each node $D^{-1}AH^{(l)}W^{(l)T} \in \mathbb{R}^{N \times d^{(l+1)}}$.

This procedure yields two $N \times d^{(l+1)}$ matrices containing a learned linear projection of each node's own features $H^{(l)}B^{(l)T}$, and a learned linear projection of each node's averaged neighborhood features $D^{-1}AH^{(l)}W^{(l)T}$. As these matrices have the same dimension and can be summed, and the element-wise nonlinearity σ can be applied over the matrix entries, we arrive at the final matrix formulation:

$$H^{(l+1)} = \sigma\left(D^{-1}AH^{(l)}W^{(l)T} + H^{(l)}B^{(l)T}\right) \in \mathbb{R}^{N \times d^{(l+1)}}$$

3.3 Graph attention

Question 3.4(a): How does aggregation occur in GATs compared to GCNs?

In graph attention (GAT), the aggregation of neighbors is weighted by pair-wise similarity (attention) scores between the node and its individual neighbors - neighbors with more similar features to the center node are given more weight in the aggregation. In graph convolution (GCN), each node's neighbors are aggregated by averaging over the features of all neighbors with equal weight.

Question 3.4(b): Dealing with bots in social networks GATs vs GCNs When building a recommendation system for a social network graph (users - nodes, social relationships - edges), GATs and GCNs would handle bots/spam accounts differently. While GAT layers would assign real user neighbor nodes with more weight (real user

nodes would have more similar features to other real users), and would assign much less weight to bot/spam neighbor nodes. This is because bot/spam accounts have atypical characteristics in how they post and behave on social media, which would be reflected in atypical node features with low similarity to real user node features, contributing to low attention weights for real-bot node-neighbor pairs. This hypothesis could easily be verified by inspecting the attention weights for some example users with known connections to both real and bot accounts. On the other hand, a GCN layer would uniformly average over all neighboring node features, which could be detrimental because the bot features might be complete outliers compared to the distribution of real user features, and thus significantly skew the naive mean aggregation by a GCN layer.

Question 3.4(c): Advantages of GATs vs Transformers for graph data tasks

One advantage of GATs over Transformers for tasks involving graph-structured data is that GATs constrain the attention operation only to neighboring nodes, providing a useful inductive bias for the network to focus only on relevant pair-wise relationships. In addition, GAT layers are compositional in nature, with successive layers growing the "receptive field" of each node similar to CNNs, which is useful for interpretability of the network. In contrast, a naive Transformer implementation might straight away compute all pair-wise attention scores even between nodes that are not connected, potentially reaching some "hacky" solutions that might generalize worse than principled attention over connected neighbors only. Furthermore, by leveraging the innate sparseness of graphs and only computing attention scores over each node's sparse connections, GATs are much more computationally efficient than Transformers which would needlessly compute all pair-wise attention scores, many of which would be computed for unconnected nodes.

Question 3.4(d): Training GCN and GAT Figure 8 shows the training and validation accuracy of GCN and GAT networks trained on the Cora dataset for 50 epochs. Note that our matrix and message-passing GCN implementations are equivalent (black and blue lines overlap exactly). The results show that GAT achieves higher validation accuracy and has a smoother learning curve compared to GCN.

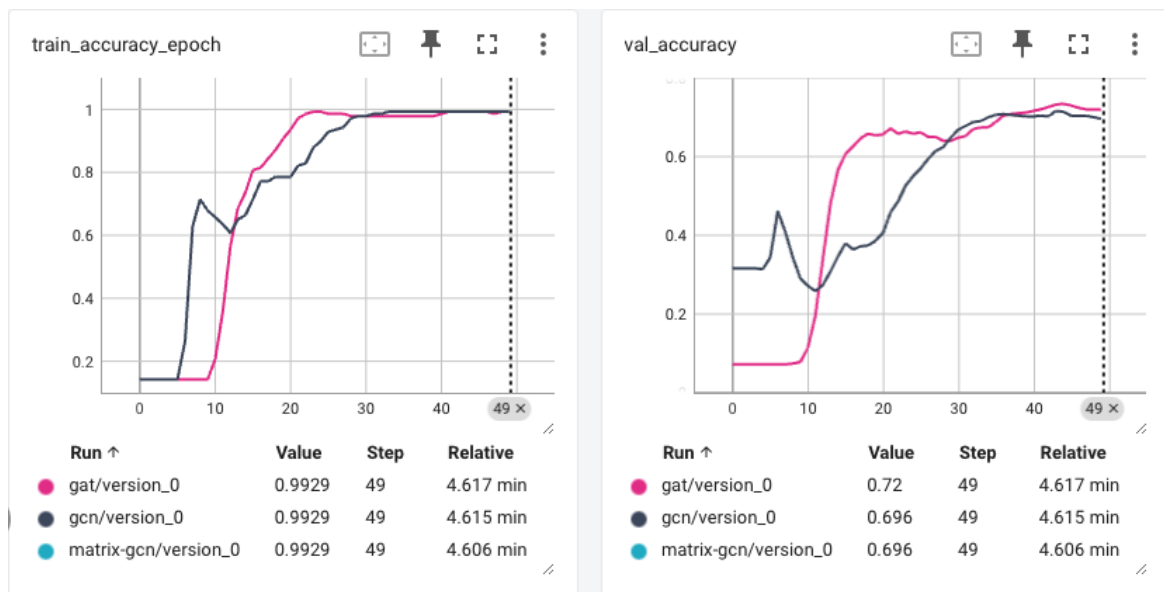


Figure 8: Training and validation accuracy of GAT vs GCN on Cora dataset.